L01: Algorithms & Analysis

Asymptotic bounds & sums

$O(g): f \le Cg$; $\Omega(g): f \ge cg$; $\Theta(g): cg \le f \le Cg$ $(0 < c \le C, n \ge n_0)$. $\Theta(g) = O(g) \cap \Omega(g)$; bounds $\neq$ best/worst input cases.

$$1 \prec \log n \prec n \prec n \log n \prec n^2 \prec 2^n \prec n!,$$
$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2} = \Theta(n^2),$$
$$\sum_{i=0}^{h} r^i = \frac{r^{h+1}-1}{r-1} \ (r \neq 1),$$
$$H_n = \sum_{i=1}^{n} \frac{1}{i} = \Theta(\log n),$$
$$\sum_{i=1}^{n} \log i = \log(n!) = \Theta(n \log n).$$

Loop templates: $\sum_{t=0}^{\lfloor \log_b n \rfloor} b^t = \Theta(n)$ $(b > 1)$; $\sum_{t=0}^{\lfloor \log_b n \rfloor} \lfloor n/b^t \rfloor = \Theta(n)$; $\sum_{i=1}^{n} (\lfloor \log_b i \rfloor + 1) = \Theta(n \log n)$. Θ **proof:** upper by max term; lower from last $\ge n/2$ terms (each $\Omega(\log n)$ for log sums).

Correctness & searching

Loop invariant: initialization $\rightarrow$ maintenance $\rightarrow$ termination; also prove progress/termination. **Linear:** best $\Theta(1)$; worst $\Theta(n)$; successful uniform average comparisons $(n+1)/2$; space $\Theta(1)$. **Binary:** sorted A; best $\Theta(1)$, worst/standard average $\Theta(\log n)$; iterative aux. space $\Theta(1)$, recursive (index-only) stack $\Theta(\log n)$.

$$m = l + \left\lfloor \frac{r-l}{2} \right\rfloor,$$
$$\begin{cases} l > r & \Rightarrow \text{return } -1, \\ A[m] = x & \Rightarrow \text{return } m, \\ A[m] < x & \Rightarrow l \leftarrow m+1, \\ A[m] > x & \Rightarrow r \leftarrow m-1. \end{cases}$$

Invariant: if x exists, it remains in $A[l \ldots r]$; $l = r$ is one candidate, $l > r$ is empty (do not index).

Sorting

Algorithm	Best	Worst	Aux.	Stable
Bubble V4	n	n^2	1	Y
Insertion	n	n^2	1	Y
Selection	n^2	n^2	1	N
Merge	$n \log n$	$n \log n$	n	Y*

*Entries are $\Theta(\cdot)$; *merge ties: take left first.* **Invariants:** Bubble: suffix contains i largest (sorted); Insertion: processed prefix sorted; Selection: prefix contains i smallest (sorted); Merge: output prefix sorted after each extraction.

$$I = \sum_{i<j} [A_i > A_j], \quad T_{\text{ins}} = \Theta(n+I), \quad C_{\text{sel}} = \frac{n(n-1)}{2},$$
$$T_{\text{merge}} = 2T(n/2) + \Theta(n) = \Theta(n \log n).$$

Special: ordered segments of length k: bubble $\Theta(nk)$ worst case, at most $k-1$ whole-array passes (known condition); $[2, 3, \ldots, n, 1]$: bubble $\Theta(n^2)$, insertion $\Theta(n)$. Two distinct keys can still give $I = \Theta(n^2)$. Selection's nonadjacent swap may break stability; Bubble V4 stops on a no-swap pass. Standard merge peak aux. space $\Theta(n)$ (reused buffer); L01 slides list $O(n \log n)$ for a different allocation/accounting convention.

Recursion tree & Master Method

For $T(n) = aT(n/b) + f(n)$ $(a \ge 1, b > 1)$, let $p = \log_b a$, $h = \log_b n$ (assume $n = b^h$, $T(1) = \Theta(1)$):

$$W_i = a^i f(n/b^i), \quad L = a^h = n^p,$$
$$T(n) = \textstyle\sum_{i=0}^{h-1} W_i + \Theta(n^p).$$

For $f(n) = \Theta(n^c)$: $W_i = \Theta\left(n^c (a/b^c)^i\right)$; $c < p$: leaves; $c = p$: equal levels; $c > p$: root. **Basic Master** (eventually nonnegative f, $\epsilon > 0$):

$$f = O(n^{p-\epsilon}) \Rightarrow T = \Theta(n^p),$$
$$f = \Theta(n^p) \Rightarrow T = \Theta(n^p \log n),$$
$$f = \Omega(n^{p+\epsilon}) \Rightarrow T = \Theta(f(n)).$$

Case 3 also requires $af(n/b) \le \rho f(n)$, $\rho < 1$. **Extended Master:** $f = \Theta(n^c \log^k n)$ (eventually positive):

$$T(n) = \begin{cases} \Theta(n^p), & c < p, \\ \Theta(n^c \log^k n), & c > p, \\ \Theta(n^p \log^{k+1} n), & c = p,\ k > -1, \\ \Theta(n^p \log\log n), & c = p,\ k = -1, \\ \Theta(n^p), & c = p,\ k < -1. \end{cases}$$

At $c = p$, $h = \log_b n$:

$$T = \Theta\left(n^p \textstyle\sum_{j=1}^{h} j^k\right).$$

$\sum j^{-1} = \Theta(\log h)$, $\sum j^{-2} = \Theta(1)$.

L01 exercise traps

Signed recurrence: standard Master does not apply to $f(n) = n \sin n$. For $n = 2^h$, $T(1) = c > 0$:

$$T(n) = n^2 \left(c + \textstyle\sum_{j=1}^{h} \frac{\sin(2^j)}{2^j}\right) = \Theta(n^2).$$

Indeed $S_2 = \sin 2/2 + \sin 4/4 \approx 0.26545 > 1/4 = \sum_{j \ge 3} 2^{-j}$, hence $0 \le S_h \le 1$. Negative terms can cancel: $4T(n/2) - n$, $T(1) = 1$ has solution $T(n) = n$.

L02: Memory Hierarchy & Cache

Memory, locality & units

Latency: time per operation; **throughput:** achieved work/time; **bandwidth:** maximum data/time. Data throughput $\le$ bandwidth; high bandwidth $\not\Rightarrow$ low latency. Faster memory is generally smaller and more expensive per byte. **Temporal:** same data soon; **spatial:** nearby data soon. Reuse after many intervening distinct blocks may miss. **CPU:** registers $\rightarrow$ L1/L2/L3 $\rightarrow$ DRAM; hardware-managed cache. **GPU:** per-thread registers; per-block on-chip shared memory (explicit); hardware L1/L2; global DRAM. Local memory is thread-private DRAM space (e.g. register spills), not necessarily fast/on-chip. Constant: broadcast reads; texture: spatial locality. **FPGA:** registers, LUTRAM, BRAM/URAM; explicitly designed buffers/dataflow; external SRAM/DRAM/Flash.

$$W(n) = \text{computational work},$$
$$Q(n) = \text{cache misses}, \quad \#\text{lines} = M/\mathcal{B}.$$

Cache holds *many* lines; a miss transfers a whole line. M: cache capacity; $\mathcal{B}$: line size (distinct from matrix B). **Units:** address mapping uses bytes; matrix analysis uses elements/words. For e bytes/element, convert both: $M = M_{\text{bytes}}/e$, $\mathcal{B} = \mathcal{B}_{\text{bytes}}/e$.

Cache mapping & address fields

Byte address a, w address bits, k ways; M is data capacity (excludes tag/valid metadata). Assume $\mathcal{B}$ and set count E are powers of two.

$$E = \frac{M}{k\mathcal{B}}, \quad q = \left\lfloor \frac{a}{\mathcal{B}} \right\rfloor,$$
$$\text{offset } o = a \bmod \mathcal{B},$$
$$\text{set } s = q \bmod E,$$
$$\text{tag } t = \lfloor q/E \rfloor = \lfloor a/(M/k) \rfloor,$$
$$a = [\text{tag} \mid \text{set index} \mid \text{offset}],$$
$$b_o = \log_2 \mathcal{B}, \quad b_s = \log_2 E,$$
$$b_t = w - b_s - b_o = w - \log_2(M/k).$$

Direct mapped: $k = 1$; unique line/slot; $b_s = \log_2(M/\mathcal{B})$, $b_t = w - \log_2 M$. k**-way:** one fixed set, any of its k ways. **Fully associative:** $k = M/\mathcal{B}$, $E = 1$, no index bits; any line, $b_t = w - \log_2 \mathcal{B}$. **Hit:** selected set contains a valid way with matching tag; offset selects byte, not hit/miss. On miss: use invalid way first, otherwise evict within the selected set; install block, tag, valid bit. **LRU:** hit or fill $\rightarrow$ MRU; evict least recently used valid line. **Example:** $w = 16$, $M = 256$ B, $\mathcal{B} = 16$ B, $k = 4$: $E = 4$, $(b_t, b_s, b_o) = (10, 2, 4)$. $a = \texttt{0x035A} = 858$: $q = 53$, $(t, s, o) = (13, 1, 10)$.

Miss types & common traps

For an actual miss, ignoring coherence: **(1) Cold/compulsory:** first access to this *block*, not each element. **(2) Capacity:** previously accessed block also misses in a same-capacity fully associative LRU reference cache. **(3) Conflict:** that reference hits, actual mapped cache misses. Compare the *same trace* and line size; reference policy here is LRU, not the optimal policy of the ideal-cache model. **Traps:** capacity misses can occur in all organizations; conflict misses occur in direct *and* set associative caches; fully associative removes mapping conflicts. A full set/way does not by itself classify the miss; first access stays cold even if it evicts another block. **Sharing/coherence:** another core's write may invalidate a cached line. True sharing: same data; false sharing: different data on the same line. Read-only sharing alone does not cause invalidation ping-pong; padding/separating writable data can reduce false sharing.

Double buffering: overlap load/compute

$N \ge 1$ equal chunks; $T_L, T_C > 0$ load/compute time per chunk. Two disjoint buffers alternate: compute current chunk while loading next; wait for both before swapping/reusing buffers. Assume independent load/compute resources, serial order within each stage, and no synchronization/switching overhead (no extra output stage).

$$T_{\text{single}} = N(T_L + T_C),$$
$$H = \max(T_L, T_C), \quad L = \min(T_L, T_C),$$
$$T_{\text{double}} = T_L + (N-1)H + T_C = NH + L,$$
$$\text{Speedup}_N = \frac{N(H+L)}{NH+L},$$
$$\text{Speedup}_\infty = 1 + \frac{L}{H} \le 2.$$

First load + steady-state overlap + final compute. $T_L > T_C$: load-bound; $T_C > T_L$: compute-bound. Balanced stages maximize speedup: if $T_L = T_C$, $\text{Speedup}_N = 2N/(N+1) < 2$; approaches 2 as $N \rightarrow \infty$. $N = 1$: no overlap, speedup 1. More buffers do not remove the slower-stage limit.

Ideal cache & caching lemmas

Ideal: fully associative, optimal replacement (future knowledge). **LRU lemma:** same access trace, line size and initially empty caches:

$$Q_{\text{LRU}}(2M) \le 2Q_{\text{Ideal}}(M).$$

LRU has *twice* the capacity; not a same-capacity bound, not equality. **Cache-Miss Lemma:** access only r contiguous segments of lengths S_i (segments need not be adjacent), $N = \sum_i S_i$; fully associative cache.

$$N < M/3, \quad N/r \ge \mathcal{B}$$
$$\Longrightarrow \quad Q \le N/\mathcal{B} + 2r \le 3N/\mathcal{B}.$$

Row/segment boundary overhead gives $+2r$; all touched lines fit, so repeated accesses hit under LRU/ideal. Bound $\neq$ exact count. **Tall cache:** $\mathcal{B}^2 < cM$ for a suitably small constant $c > 0$; asymptotically $M = \Omega(\mathcal{B}^2)$. **Submatrix Caching Lemma:** row-major $n \times n$ submatrix; each row is a contiguous segment ($N = n^2$, $r = n$):

$$\mathcal{B}^2 < cM \le n^2 < M/3$$
$$\Longrightarrow \quad Q \le 3n^2/\mathcal{B}.$$

Lower condition ensures $n > \mathcal{B}$ (long rows); upper condition reserves room for row-edge lines. These are sufficient conditions; smaller submatrices can also cache efficiently.

Naive matrix multiplication: three regimes

Assumptions: row-major A, B, C; loops $i \rightarrow j \rightarrow k$; fully associative LRU, tall cache, cold start; M, $\mathcal{B}$ in elements.

$$C_{ij} \mathrel{+}= A_{ik} B_{kj}, \qquad W(n) = \Theta(n^3).$$

Inner k: A contiguous, B stride n, C_{ij} reused. For $n \ge \mathcal{B}$, one B column touches n distinct lines; holding these lines needs about $n\mathcal{B}$ elements, not n.

Size condition	$Q(n)$
$n > c_1 M/\mathcal{B}$	$\Theta(n^3)$
$c_2\sqrt{M} < n < c_3 M/\mathcal{B}$	$\Theta(n^3/\mathcal{B})$
$n < c_4\sqrt{M}$	$\Theta(n^2/\mathcal{B})$

c_i are suitable constants, separately for each regime; the two threshold *scales* are $\sqrt{M}$ and $M/\mathcal{B}$, not exact numerical cutoffs (transition bands omitted). Cold-cache expressions above assume $n^2 \ge \mathcal{B}$; for tiny matrices use $\Theta(1 + n^2/\mathcal{B})$ in the fitting case. **Large:** n current B lines do not fit; little reuse between columns; fixed i: $\Theta(n^2)$ misses, then n values of i. **Middle:** these n lines fit, entire B does not. Each group of $\mathcal{B}$ adjacent columns loads n lines; fixed i: $(n/\mathcal{B})n = n^2/\mathcal{B}$; multiply by n outer iterations. **Small:** all three matrices fit; load each line once, reuse across all i. Ignoring row-edge overhead, $3n^2 \le M$ is the simple capacity check. For $n = 16$, $\mathcal{B} = 4$, cold B alone needs $256/4 = 64$ lines, not 256; no further factor n if it remains cached.

Blocking / tiling: short derivation

Supplementary derivation from the lecture's blocking idea. $S \times S$ tiles, $\mathcal{B} \le S \le n$; assume $S \mid n$ for counts, fully associative cache, and all three actual tile line footprints fit. Tile rows are contiguous; row-edge overhead is $O(S)$ lines.

$$W_{\text{tile}} = \Theta(S^3), \quad D_{\text{tile}} = \Theta(S^2),$$
$$Q_{\text{tile}} = O(S^2/\mathcal{B} + S) = O(S^2/\mathcal{B}),$$
$$\#\text{tile products} = (n/S)^3,$$
$$Q_{A+B} = O\left((n/S)^3 S^2/\mathcal{B}\right) = O(n^3/(\mathcal{B}S)).$$

Keep C **resident:** outer tile indices I, J, inner K; $C_{IJ} \mathrel{+}= A_{IK} B_{KJ}$ for every K before releasing C_{IJ}.

$$Q_C = O\left((n/S)^2 S^2/\mathcal{B}\right) = O(n^2/\mathcal{B}),$$
$$Q_{\text{blocked}} = O\left(\frac{n^3}{\mathcal{B}S} + \frac{n^2}{\mathcal{B}}\right).$$

Ignoring row-edge overhead: $3S^2 \le M$. For large matrices, choose $S = \Theta(\sqrt{M})$ with a small enough constant to leave line-boundary headroom; tall cache ensures $S \ge \mathcal{B}$.

$$Q_{\text{blocked}} = O\left(\frac{n^3}{\mathcal{B}\sqrt{M}} + \frac{n^2}{\mathcal{B}}\right).$$

Work stays $\Theta(n^3)$; reuse per loaded tile grows as $\Theta(S)$. Too-large tiles evict useful lines; strided access alone does not force a miss if the needed line is still cached. Write-back adds only a constant factor here.